Department of Electrical and Computer Systems Engineering
ECE2071 - Systems Programming

ECE2071 Project

Welcome and Getting Started

Welcome to the ECE2071 project, in which you will build an audio recording and processing embedded system. The project is broken down into an initial milestone, followed by 4 tasks that build upon each other to produce the final result, which you will present in week 10.

- Initial Milestone: Explore inter-device communication necessary for project
- Task 1: Create a minimum viable product version of the system
- Task 2: Implement a user interface and different capture modes
- Task 3: Improving the audio quality that is captured
- Task 4: Further audio improvements (TBA)
- Presentation: Demonstrate your design, explain your design process, and reflect on how you planned and delivered the project.

As you complete the labs, and begin to work on your solution with your team, you should frequently come back to the detailed specification as required to ensure your design meets all of the requirements (this is to give you experience with implementing complex requirements).

The document also includes the marking rubric that explains the criteria on which you will be graded for this assessment.

Table of Contents

ECE2071 Project **1**
Welcome and Getting Started 1
Table of Contents 2
Revision History 3
Introduction 4
Learning Outcomes 4
Academic Integrity and AI Use: 4
Submission 5
Presentation 5
Milestone (5%) 5
Task 1: Minimum Viable Product (6%) 6
Task 2: User Interface and Modes (2.5%) 8
Task 3: Audio Improvements (5%) 10
Notes: 10
Task 4: Further Audio Improvements (2.5%) 12
Presentation and Project Management (4%) 12
Marking Scheme for Design Demonstration (16%) 13
Marking Scheme for Presentation and Project Management (4%) 14
Rubric for individual viva-voce (scales the total project grade): 14

Revision History

Use this revision history to monitor for any changes we may need to make to this document to clarify requirements or provide hints

Description	Version	Date
Initial release	v1.0	14/03/2026

Introduction

This project aims to develop an proximity triggered data acquisition system that records and analyses audio samples, making use of STM32 microcontrollers. This will be achieved via both guided and unguided laboratory activities that introduce key components, and an initial milestone to demonstrate vital basic functionality that 'unlocks' the remainder of the project.

The final system design will combine two STM32 microcontrollers; the first will sample audio data from its onboard analog-to-digital converter (ADC), while the second will provide proximity detection and some basic audio processing. The data captured from this process will be transmitted to a personal computer via USART. The processed audio samples will then be interpreted & stored using a combination of Python script and C program.

This project is designed to be completed incrementally, with each of the constituent tasks building on the previous one, adding functionality and improving overall system performance. We've broken the project into smaller, individually testable, components with specific performance requirements to aid you in verifying and validating performance as you go, and to give teams the option of deciding how much of the project they wish to attempt.

Learning Outcomes

- Construct an embedded program that integrates multiple common microcontroller peripheral functionalities together into an effective system.
- Use interrupts to precisely control timing behaviour in embedded systems
- Utilise number representations to efficiently transmit data between embedded systems using standard serial communications protocols
- Develop and reflect on project management plans for systems, including the use of block diagrams and software version management (git).

Academic Integrity and AI Use:

You may discuss **general** details (eg: approaches) of the project with students from other teams. However, **you may not:**

- write the code together with students from other groups,
- show students from other groups your code,
- copy other group's work, or
- present work you have found online as your own (i.e. without attribution),

This is plagiarism and/or collusion, and is prohibited by Monash's academic integrity policy. Breaches of this policy attract serious consequences.

If you research some concept online or re-use code from your labs (or from this unit) you must cite it (any citation method is acceptable).

Generative AI

In this assessment, you may use generative AI in order to develop your understanding of your code (eg "explain how this works to me", or "help me find the bug in this code") only.

You may not use it to directly generate programs for submission. Any use of generative AI for this purpose must be explicitly acknowledged in a comment, explaining your prompts and how you used the outputs. If you do not acknowledge your use of AI, it may be submitted as plagiarism under the academic integrity guidelines.

Any work submitted for a mark must:

- represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
- adhere to the guidelines for AI use set for the assessment task.
- reflect the University's commitment to academic integrity and ethical behaviour.
- Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

Submission

This project consists of two submissions:

1. The project milestone is due for demonstration during your allocation lab session in Week 6. No work needs to be submitted ahead of the class for this assessment component.
2. The final project submission is due at 11:55 pm on Week 10 Monday, 11 May. Submissions are to be made by group and should include your presentation slides, a recording of your presentation (with maximum duration of 5 minutes), a zipped copy of your project code, and any video evidence you wish to include on the functionality of the prototype.

Special Consideration:

Special considerations are applied on a per student basis. If approved, you will individually be assigned an alternative assessment.

Presentation

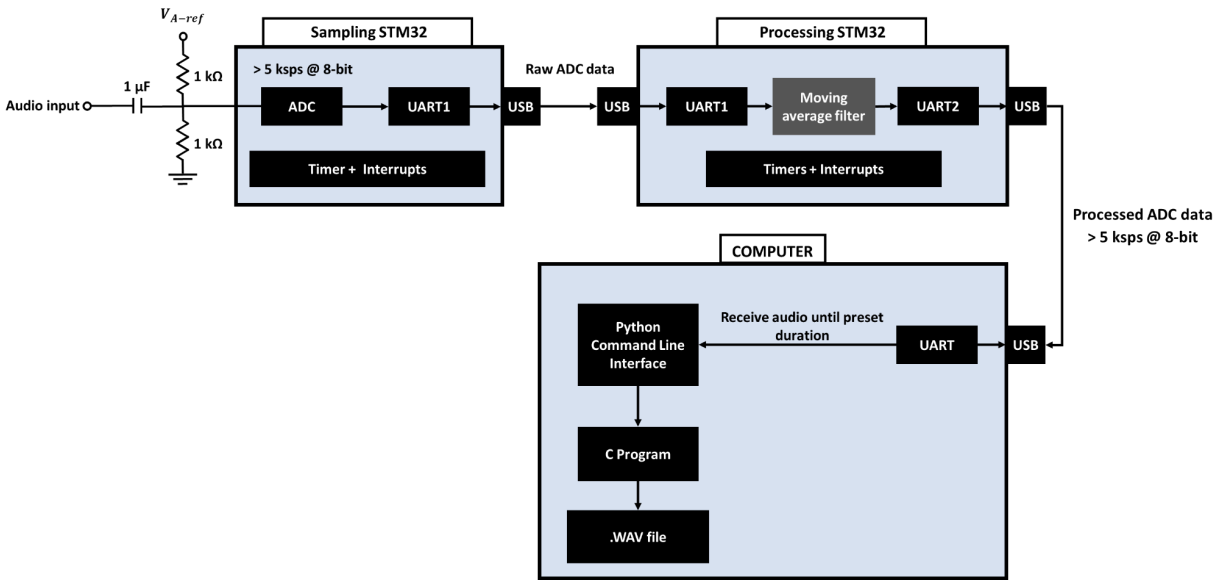
Your group will need to demonstrate your design to the unit coordinator during your allocated Week 10 lab session. The aspects to include in this demonstration are included in this specification document.

Milestone (5%)

The project milestone requirements are specified in a separate document available here:

ECE2071 Project S1 2026 - Week 6 Milestone (Malaysia Campus)

Task 1: Minimum Viable Product (6%)



Your company's leadership was impressed with the investigation your team completed into STM networking, and has given you a new objective to design a dual-STM audio processing system.

Your first task towards this product is to create a 'Minimum Viable Product' (MVP) version of the final desired system. By concentrating on only the core essential functionality first, we can verify the system's fundamental objectives first. This approach will also allow us to check and troubleshoot the impact of additional functions as we add them to the system.

For this data acquisition system, the MVP can be broken down into three key subsystems:

- **The 'Sampling' STM32**: This subsystem captures audio samples from the ADC and transmits the data to the next STM32.
- **The 'Processing' STM32**: This subsystem receives the raw samples from the Sampling microcontroller, processes the audio data, and transmits it to a PC for analysis and storage.
- **The Python Script**: This is a script written to communicate with the 'Processing' microcontroller using the STM's inbuilt USART to perform the following activities:
    - Initially receive the processed audio samples using the Python *pyserial* module and save as a .txt file
    - Compile a C program and run the generated executable using the Python *subprocess* module to convert the audio samples into a .WAV file for future playback

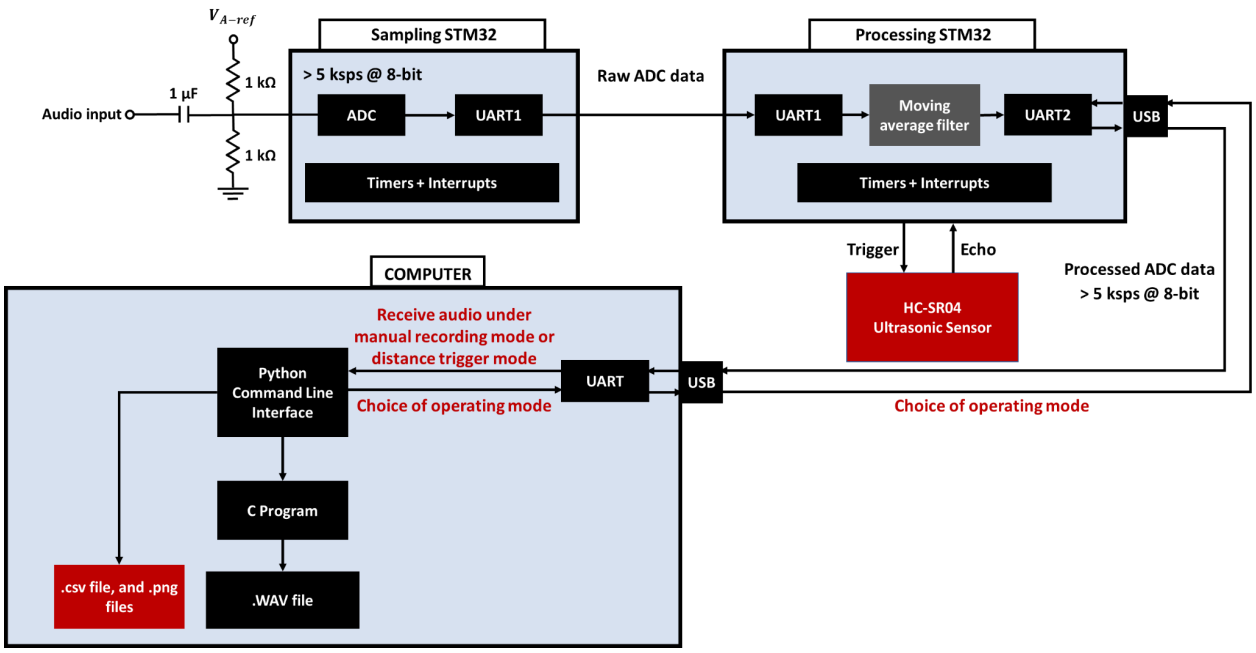
As this is only the minimum viable product, your manager has instructed you to not worry about aspects of the final project such as good user interface design, precise choice of sampling rate, or audio quality.

The items we wish your design to deliver for this task are as follows:

- Audio samples captured by the ADC on the 'Sampling' STM should contain at least 8 bits.
- The rate at which audio samples are taken should be at least 5ksps (5,000 samples per second)
- The 'Processing' STM should implement a 'moving average filter' of at least length of 3 on the data it receives from the 'Sampling' STM.
- A Python script should be used to read audio samples from the 'Processing' STM.
- The length of the audio recording should be hard-coded into the Python script.[1]
- The Python script should compile and run a C program to convert the audio samples into a .WAV' file.
- The system should consist of 2 STM32 microcontrollers

[1]It is not required for the python script to 'talk back to' the STM at this point. Remember that UART only transmits data when the receiver is ready. This can be used to control the flow of data in this simple setup.

Task 2: User Interface and Modes (2.5%)



Now that the viability of the project has been proven, your manager has requested that you expand the system by building out a proximity trigger and a command line interface (CLI) for the product. The CLI will also enable more in depth testing.

The distance (aka proximity) trigger feature to be added should enable the system to start (and stop) recording audio automatically when a user is within a configurable distance. The CLI program should be created by expanding on the initial Python script developed for Task 1 to allow for signal analysis and the multiple output options as described below.

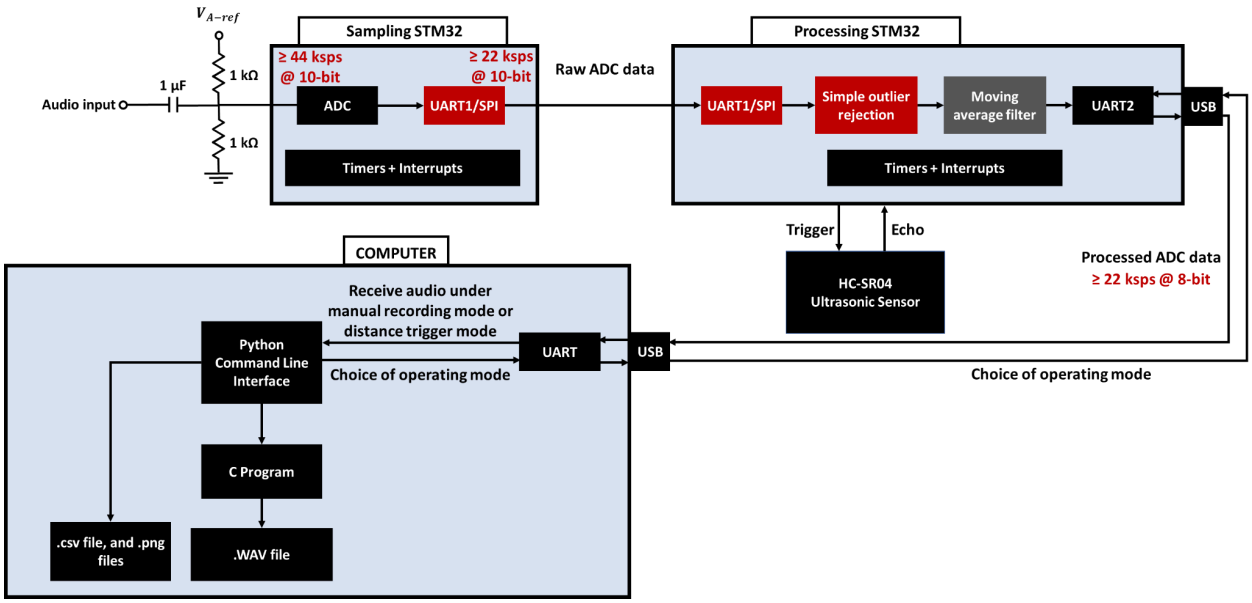
The exact details about how you will implement both the user interface and the proximity trigger <u>are left to you and your team</u>, but some key requirements are specifically required:

- The CLI should include a clear menu structure and be well documented, so it is clear what the options are for each operating mode, as well as what the CLI expects the user to do at any given time.
- Your system should consist of two operating modes, that may be swapped between:
    - 'Manual Recording Mode': This mode will allow the user to specify how long to record for in the CLI (i.e. it will no longer be hardcoded) as well as generate any of the output formats as requested by the user.
    - 'Distance Trigger Mode': This mode will allow the 'Processing' STM to control when data is to be recorded using an ultrasonic sensor, with options to specify the user's requested output format(s).

        - The ultrasonic sensor should reliably begin recording when it detects an object within a custom range, although the client has suggested 10cm as a default distance.
        - When the object is no longer detected, the recording should reliably stop after a short interval time[2].
        - The system should continue to trigger from the sensor as long as the CLI remains in this mode.
- Your system must be capable of outputting the audio data in a range of formats[3];
    - A '.wav' file of the received audio samples (created by compiling the relevant C program and running the generated executable) to enable playback.
    - A '.png' file showing a plot of the 'amplitude' vs 'time' waveform. The plot produced should include automatic titles and axis labelling, as the output should be of a standard that is ready for a user to immediately interpret.
    - A '.csv' file showing the processed audio data, ensuring the first row indicates the sample rate of the signal so it may be reconstructed or processed further by the user.

[2] The behaviour of the ultrasonic sensor can generate momentary spikes in distance, and therefore it is important you account for this 'bouncing' in your logic.
[3] Each file should be appropriately named, and contain your team's ID and the sampling rate of the audio.

Task 3: Audio Improvements (5%)



Your manager has observed that the base audio quality provided by the minimum viable product is 'somewhat lacking'. While this is expected, it's time to fix this. You have therefore been instructed to enhance the system's quality, bringing it up to a level that would be acceptable in a commercial product.

When we consider 'audio quality', we often consider two key factors: the 'bit-depth' of the audio, which is the number of bits per sample, and the 'sample rate' of the audio, which is the number of samples per second. This means that to improve the quality of your audio recordings, you must either record more bits per sample or speed up the rate at which your system is sampling. This represents a challenge for embedded systems, which are characterised by a limit to the amount of available processing power to execute the required steps at faster and faster rates.

- Your system should now **sample at a minimum rate of 44 ksps, with a bit-depth of 10 bits per sample.**
- Your system should **downsample this audio data to 22 ksps with a bit-depth of 8 bits per sample** before transmitting it to the connected PC, to ease the burden on the connection between the Processing STM and the PC.
- Your system should, via the Processing STM, perform a simple outlier rejection* in addition to an optimised moving average filter to improve the output sound quality.

Notes:

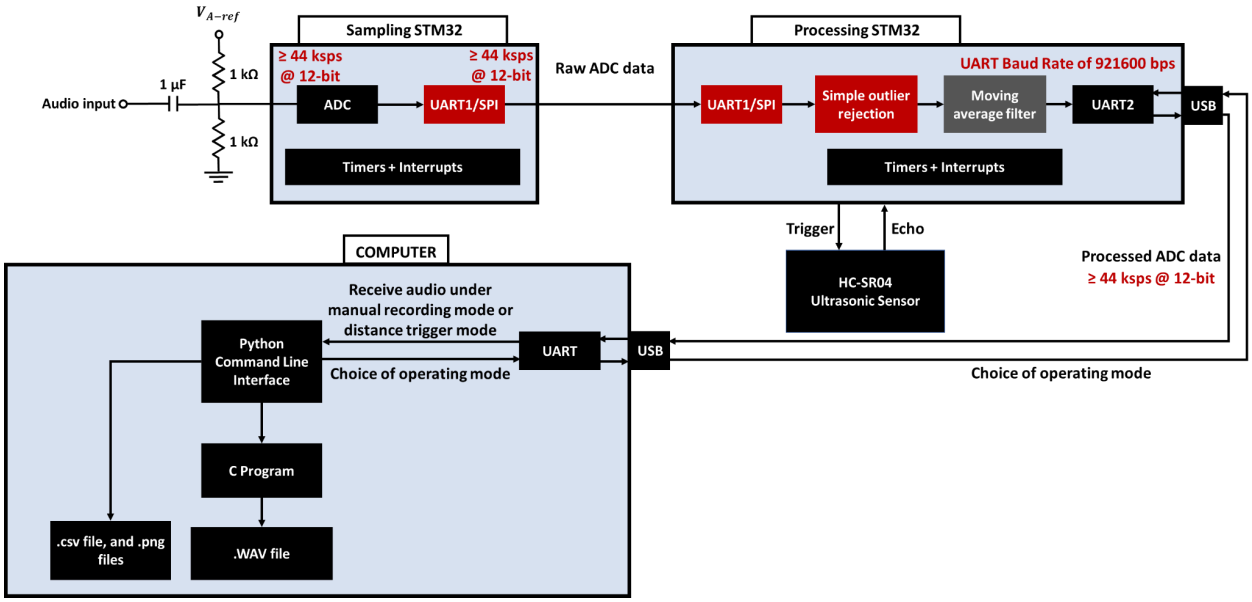
As a reminder, you may use any serial protocol you wish, and may need to change the protocol you used in the milestone.

The exact method you use to downsample/downscale is your decision, but keep in mind that improving audio quality is the primary objective of this task, and that you can adjust the length of

the moving average filter from Task 1 to improve performance. You can downsample the sampling rate on the sampling STM if you wish. However, rescaling of the ADC data from 10-bits to 8-bits should be done on the processing STM.

*A simple outlier rejection algorithm would be to remove values that are a tunable distance away from the mean of the signal. This distance should be tweaked to optimise the sound quality of the output. You are welcome to experiment with more complex algorithms if you wish, but remember the limitation of processing power in this environment.

Task 4: Further Audio Improvements (2.5%)



Your manager is pleased with the product delivered in Task 3. For the final task, we want to push the scope of what our system achieves, in order to have a more compelling offering in the market. To meet the requirements of this task, you will need to conduct your own research and independently redesign key components of the existing hardware and software subsystems.

The objectives of this task are as follows:

- The final output audio file should be at a sampling rate of 44 ksps or higher
- The bit-depth of the final audio file should be 12-bit
- Your system **must** use an industry-standard baud rate to communicate with the PC, which should be no higher than 921600 bits/second.

Notes:

This task is designed to be a challenge for students seeking an HD, and will require significant effort[4]. We are not providing detailed guidance on how to approach this problem, as this is part of the challenge, although we're happy to discuss ideas with you.

In order to help your system reach these requirements, we recommended that you look into the following topics:

- Serial Peripheral Interface (SPI)
- Direct Memory Access (DMA)
- Error detection for serial data communication

[4] We have verified this task is possible.

Presentation and Project Management (4%)

You should upload a video recording of your presentation (with a maximum duration of 5 minutes) along with the material used for the presentation by 11:55 pm on Week 10 Monday. The presentation will be graded against the standard expected of a graduate engineer. It must be well thought out and prepared.

Your presentation will be marked based on the following criteria:

1. **Professionalism, preparedness, and overall quality**
    a. The presentation is well prepared and well organised.
    b. All team members contribute to the presentation of your product.
    c. The presentation is accompanied by audio and visual elements that **support** the content being presented.

2. **Flow of presentation and content presented**
    a. The presentation flows smoothly through each section, making it easy for your manager to understand the choices you've made and the decisions you've taken.
    b. The presentation clearly demonstrates all of the key aspects of the project as completed by the team in a logical order.

3. **Project management and planning**
    a. The presentation discusses how the team made effective use of block diagrams to describe and design the system before code was written. The development of block diagrams from the initial concept to the final version is shown.
    b. The presentation discusses how the team effectively used software version management (git) while working across the project including example commits from the development cycle.
    c. The presentation clearly describes the division of work amongst team members

Marking Scheme for Design Demonstration (16%)

During your allocated week 10 lab session, you should demonstrate your prototype to the unit coordinator, highlighting the work you and your team have accomplished. The demonstration is worth 16% of your final grade.Each item on this list will be marked on a pass/fail basis.

Task 1: Minimum Viable Product

The system (from acquisition through to output files) operates continuously at:	
A sample rate of greater than 5 ksps	1.5
an audio resolution of at least 8 bits/sample	1.0
'Processing' STM performs a moving average filter with at least 3 sample window size	0.5
A python script reads and saves the serial output of the processing STM to file	1.0
The recording time is hardcoded into the Python script	0.5
The system output is recognisably the audio sample played at the input	1.5
Total for Task 1:	6.0

Task 2: User Interface Design & Modes

The STM can operate in 'Distance Trigger' mode	1.0
Python CLI can configure settings, switch modes, analyse and save data[5]	1.0
'Distance Trigger' mode sets recording time automatically	0.5
Total for Task 2:	2.5

Task 3: Audio Improvements

The 'Sampling' STM samples at a minimum of 44k samples/second	1.0
The 'Sampling' STM transmits 10-bits/sample at a minimum of 22k samples/second	0.5
The 'Processing' STM removes outliers from the data	1.0
The 'Processing' STM rescales the 10-bit resolution to fit an 8-bit sample	0.5
The 'Processing' STM sends output data at a rate of 22 ksps	1.0
The system output audio is comparable in quality to the computer's output	1.0
Total for Task 3:	5.0

Task 4: DMA and Further Audio Improvements

The system should operate at a sample rate of 44 ksps throughout[6]	1.5
The system should operate at a 12-bit resolution throughout[7]	1.0
Total for Task 4:	2.5

Total for Design Demonstration	**16**

[5] Achievement of this criteria will also award points for the hardcoded recording time in Task 1
[6] Achievement of this criteria will also award points for the sampling rate items in Task 3
[7] Achievement of this criteria will also award points for all prior bit depth items in Task 3

Marking Scheme for Presentation and Project Management (4%)

Each item on this list will be graded on a 4 point scale of Excellent (100%), Good (75%), Passing (50%), Below Passing (0%).

Presentation and Project Management	
The presentation was rehearsed and confident	0.5
The presentation included relevant audio and visual aids	0.5
Ordering of the project made logical sense	0.5
The presentation clearly displayed all key aspects of the final product	1
The presentation included a well-laid-out block diagram	0.5
The team used Git in a helpful and constructive way	0.5
The presentation included a clear summary of the division of labour amongst team members	0.5
Total for Presentation and Project Management:	**4.0**

Rubric for individual viva-voce (scales the total project grade):

Category Description	Detailed Description	Scaling Factor
Expert understanding	The student is in full command of every aspect of the design, and demonstrates a **deep familiarity of the design.** They can answer questions precisely, correctly, and concisely with no prompting.	100%
Complete understanding	The student is **well prepared** and can **consistently provide answers** that are **fully correct**, possibly with some prompting. The student may lack confidence or speed in answering.	90%
Tolerable understanding	The student is **reasonably prepared**, and gives answers that are **partially correct, or fully correct in one area,** but cannot answer questions about another area correctly.	80%
Selective understanding	The student has likely seen the design, and can answer **something partially relevant or correct** to a question but they **can't engage in a serious discussion of the work**.	70%
Trivial understanding	The student may have seen the design/circuits before, and can answer **something partially relevant or correct** to a question but they **clearly can't engage in any discussion of the work**.	60%
No Understanding	The student has **not prepared**, **cannot answer even the most basic questions** and likely has not even seen the design/circuits before, or did not attend the interview.	0%

Note: Post viva-voce scaling of the total project grade for each team member, further rescaling will be conducted via peer-assessment through ITP metrics